

# Relatório 1º projeto ASA 2025/2026

Grupo: AL096

Aluno(s): Bruna Almeida (114305) e João Varanda (114010)

## Descrição do Problema e da Solução

O problema consiste na participação num jogo de realidade virtual onde é retirado, vez à vez, um aminoácido contido numa cadeia de  $n$  aminoácidos de uma proteína, provocando a libertação de energia. A energia que cada aminoácido liberta depende do seu potencial de estabilidade, do potencial de estabilidade dos aminoácidos que interagem diretamente com este e da afinidade entre as suas respetivas classes bioquímicas. O objetivo principal é obter o maior valor possível de energia libertada.

De modo a auxiliar o João Caracol, foi desenvolvido um algoritmo que permite descobrir a ordem de remoção dos aminoácidos para maximizar a energia libertada no final do jogo. Esta solução implementa programação dinâmica sobre intervalos para calcular, para cada subintervalo  $[i, j]$ , qual o aminoácido removido por último e qual o máximo de energia que este permite acumular. Assim, o algoritmo reconstrói a ordem de remoção que maximiza a energia libertada através de uma pilha que atravessa a matriz que contém o último aminoácido removido para cada intervalo.

## Análise Teórica

- Leitura dos dados: Inicialmente o programa lê a quantidade de aminoácidos presentes na proteína ( $n$ ), os potenciais de estabilidade de todos os aminoácidos (vetor *stabilityPotentials*) e a classe bioquímica destes (vetor *biochemicalClasses*). Complexidade: leitura de um único valor  $O(1)$ , leitura de 2 vetores  $O(n)$ . Complexidade total:  **$O(n)$** .
- Construção da matriz de energia máxima: A matriz armazena os valores de energia máxima para cada subintervalo de aminoácidos da sequência. Complexidade: leitura de todos os intervalos possíveis de tamanho máximo  $n$   $O(n^2)$ , cálculo da energia máxima libertada pelos mesmos e do último aminoácido a ser removido para tal,  $O(n)$ . Complexidade total:  **$O(n^3)$** .

### Preenchimento da tabela de energias

```
1: function FILLMAXENERGYTABLE(n, MaxEnergy, LastRemoved, P, C)
2:   for length = 1 to n do
3:     for i = 0 to n - length do
4:       j ← i + length - 1
5:       best ← 0
6:       bestIndex ← 0
7:       for idx = i to j do
8:         leftEnergy ← MaxEnergy[i][idx-1] if idx > i else 0
9:         rightEnergy ← MaxEnergy[idx+1][j] if idx < j else 0
10:        middleEnergy ← COMPUTENERGY(i-1, idx, j+1, P, C)
11:        total ← leftEnergy + middleEnergy + rightEnergy
12:        if total > best or (total = best and idx > bestIndex) then
13:          best ← total
14:          bestIndex ← idx
15:        end if
16:      end for
17:      MaxEnergy[i][j] ← best
18:      LastRemoved[i][j] ← bestIndex
19:    end for
20:  end for
21: end function
```

# Relatório 1º projeto ASA 2025/2026

Grupo: AL096

Aluno(s): Bruna Almeida (114305) e João Varanda (114010)

- Obtenção da ordem de remoção de aminoácidos que maximiza a energia libertada. Complexidade: análise da matriz e organização da ordem através de uma pilha  $O(n)$ . Complexidade total:  $O(n)$ .

```
Reconstrução do vetor de ordem de remoção
1: function GETREMOVALORDER(n, LastRemoved)
2:   initialize Order as empty list
3:   initialize Stack with element ((0, n - 1), false)
4:   while Stack not empty do
5:     (interval, shouldAdd) ← top of Stack
6:     pop Stack
7:     (i, j) ← interval
8:     if i > j then
9:       continue
10:    end if
11:    k ← LastRemoved[i][j]
12:    if shouldAdd = true then
13:      append (k + 1) to Order
14:    else
15:      push ((i, j), true) onto Stack
16:      push ((k + 1, j), false) onto Stack
17:      push ((i, k - 1), false) onto Stack
18:    end if
19:  end while
20:  return Order
21: end function
```

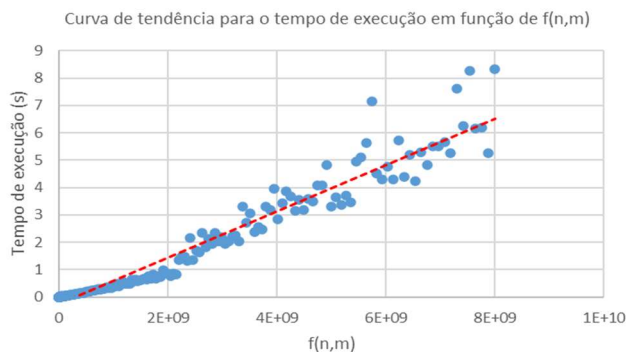
- Apresentação do resultado: São apresentados o valor máximo de energia máxima libertada pela sequência de aminoácidos e a ordem lexicograficamente menor de remoção dos aminoácidos que permite obter esse valor máximo de energia. Complexidade: apresentação do valor máximo de energia libertada  $O(1)$  e da ordem dos aminoácidos organizados no vetor *order*  $O(n)$ . Complexidade total:  $O(n)$ .

Assim, a complexidade global do algoritmo é  $O(n^3)$ , pois sobrepõe-se às outras complexidades inferiores. Todas as outras operações, contribuem com uma complexidade com uma velocidade de crescimento inferior.

Dado que são armazenadas 2 matrizes de tamanho  $n \times n$  a complexidade espacial é  $O(n^2)$ .

## Avaliação Experimental dos Resultados

Para realizar esta avaliação foram realizados 200 testes, variando  $n$  de 10 a 2000 com incrementos de 10, e com um valor de  $m$  constante igual a 20.



| $f(n, m)$  | Tempo (s) |
|------------|-----------|
| 1000       | 0.004     |
| 1000000    | 0.002     |
| 32768000   | 0.014     |
| 157464000  | 0.057     |
| 438976000  | 0.159     |
| 912673000  | 0.328     |
| 1000000000 | 0.383     |
| 3241792000 | 2.252     |
| 4913000000 | 4.84      |
| 8000000000 | 8.334     |

O gráfico obtido apresenta um comportamento linear quando o tempo de representado em função de  $n^3$ , confirmando assim a hipótese de que o algoritmo cresce sobre de acordo com uma relação cúbica.